Possíveis perguntas técnicas da banca

FATECalçados · Programa de fidelidade · Desafio de Inovação Data System 2026

Material de preparação para a apresentação. Cada pergunta vem com uma resposta sugerida, curta e fiel ao que está no código. Os nomes de arquivo servem para abrir o código na hora, caso a banca peça para ver.

---

1. Visão geral e arquitetura
2. Autenticação e sessão
3. Regras de pontos e banco de dados
4. PDV offline e idempotência
5. Animações no frontend
6. Testes e qualidade
7. Segurança e LGPD
8. Limitações e próximos passos

1 Visão geral e arquitetura

P. Qual é a stack e por que escolheram essas tecnologias?

> Backend em **Django + PostgreSQL**, frontend em **React + Vite + TypeScript**, tudo subindo com **Docker Compose**. Django já traz ORM, migrações, painel `/admin`, autenticação e proteção CSRF prontos, o que nos deixou focar nas regras do programa. PostgreSQL garante transações e travas de linha (`select_for_update`), necessárias para saldo de pontos. React com TypeScript em modo estrito pega erros de contrato da API na compilação.

P. Como o backend está organizado?

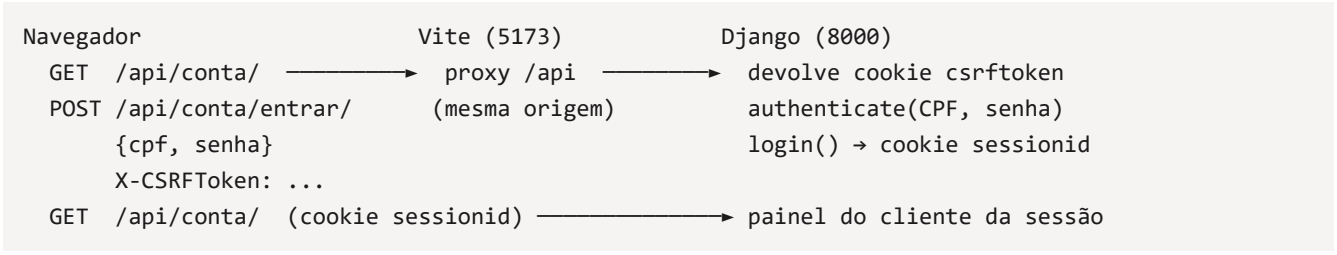
> Em camadas, e cada camada só chama a de baixo:
>
> - `regras.py`: todos os números do programa (níveis, bônus, 100 pontos = R$ 5,00, validade de 12 meses) como funções puras, sem banco.
> - `services.py`: único lugar onde as regras tocam o banco (registrar venda, expirar pontos, cadastro, login).
> - `consultas.py`, `indicadores.py`, `acoes.py`: somente leitura (listas, dashboard, ações do gerente).
> - `views.py`: só lê a requisição, chama o serviço e devolve JSON. Nenhuma regra de negócio fica aqui.
>
> Vantagem: uma mudança de regra começa em `regras.py` e é testada sem banco.

P. E o frontend?

> React Router com duas áreas: a da loja (`/admin/...`: PDV, Dashboard, Clientes, Vendas, Ações, Avaliações) e a do cliente (`/login`, `/loja`, `/minha-conta`). Estado que precisa sobreviver à troca de tela fica em *providers* de contexto (`CaixaProvider`, `ContaProvider`...). Lógica de tela fica em *hooks*, e `services/api/` é o único lugar que chama a API, traduzindo os campos em português para os nomes usados nos componentes.

2 Autenticação e sessão

> **Resumo em uma frase:** o cliente entra com CPF e senha; o Django confere a senha (guardada só como hash) e cria uma sessão no servidor, identificada por um cookie; toda requisição que altera dados precisa também de um token CSRF.



P. Como funciona o login do cliente?

> Login por **CPF + senha**, sem e-mail. Cada `Cliente` tem um vínculo um-para-um (`Cliente.usuario`) com um `User` do Django cujo *username* é o CPF. Esse `User` serve só para guardar o hash da senha e usar o sistema de autenticação do Django. Em `services.autenticar_cliente` chamamos `authenticate()` e, se der certo, `login()`, que grava a sessão.

P. Por que sessão com cookie e não JWT?

> - O front e a API ficam na **mesma origem** (o Vite repassa `/api` ao Django), então o cookie funciona sem configurar CORS.
> - O cookie de sessão é **HttpOnly**: um script malicioso na página não consegue lê-lo, ao contrário de um token guardado em `localStorage`.
> - A sessão fica no servidor, então **sair invalida de verdade** (`logout()` apaga a sessão). Com JWT seria preciso lista de bloqueio ou esperar o token expirar.
> - É o mecanismo nativo do Django, testado e mantido pela comunidade: menos código nosso para errar.

P. Como a senha é guardada?

> Nunca em texto puro. `set_password` do Django gera um hash **PBKDF2-SHA256 com salt** e milhares de iterações (padrão do Django). Antes de salvar, a senha passa pelos validadores de `AUTH_PASSWORD_VALIDATORS` (tamanho mínimo, semelhança com os dados do usuário etc.). A troca de senha (`services.alterar_senha`) exige a senha atual e usa `update_session_auth_hash` para não derrubar a sessão de quem trocou.

P. E o CSRF? Como o front manda o token?

> As rotas de `/api/conta/` usam cookie de sessão, por isso **não** são `csrf_exempt`. O `GET /api/conta/` tem `@ensure_csrf_cookie`, que entrega o cookie `csrftoken`. O `http.ts` do front lê esse cookie e o devolve no cabeçalho `X-CSRFToken` em todo POST (se ainda não tiver o cookie, faz o GET antes). Como a requisição chega com `Origin` da porta 5173, essa origem está em `CSRF_TRUSTED_ORIGINS`.

P. Como funciona o cadastro? E se o cliente já comprou na loja física?

> `services.criar_acesso` valida os dígitos do CPF e trava a linha do cliente (`select_for_update`) para evitar dois cadastros simultâneos. Se o CPF é novo, cria o cliente na "Loja Online". Se o CPF já existe (cliente que comprou no caixa), **só liga o acesso se o primeiro nome conferir** com o cadastro, para que alguém não "sequestre" os pontos de outra pessoa sabendo só o CPF. Se o CPF já tem senha, recusa.

P. O que impede um cliente de comprar ou ver dados em nome de outro?

> O CPF da compra online **sempre vem da sessão**, nunca do corpo da requisição (`services.comprar_online`). Os detalhes de compra da conta só devolvem compras do cliente logado. No front, o `ClienteLayout` manda para `/login` quem não tem sessão, mas isso é só conveniência: **quem protege é o servidor**, que responde 401 sem sessão válida.

P. A mensagem de erro do login revela se o CPF existe?

> Não: o login responde sempre "CPF ou senha incorretos". O cadastro, por outro lado, avisa "Esse CPF já tem acesso", o que permite descobrir se um CPF tem conta. Aceitamos isso pela usabilidade; uma melhoria seria limitar tentativas por IP (*rate limiting*).

P. E a área da loja (`/admin`)? Tem login?

> Ainda não, e é bom admitir isso. A área da loja só é acessada digitando a URL, e as rotas da API da loja são `csrf_exempt`, pensadas para o caixa. O próximo passo é exigir login da equipe (o Django já tem `is_staff` e grupos/permissões) e proteger essas rotas com o mesmo esquema de sessão + CSRF da área do cliente. O painel `/admin` do Django já exige usuário da equipe.

3 Regras de pontos e banco de dados

P. Onde fica o saldo de pontos do cliente?

> Não existe um campo `saldo`. Os pontos são um **livro-razão**: cada acúmulo, resgate e expiração é uma linha em `MovimentoPontos`, e o saldo é sempre a soma delas. Assim o histórico é auditável e o extrato do cliente sai direto da tabela, sem risco de o saldo "descolar" do histórico.

P. Como funcionam validade e resgate?

> Cada compra gera um **lote** que vence em 12 meses. O resgate consome os lotes do mais antigo para o mais novo (**FIFO**, `_consumir_lotes`), para o cliente perder o mínimo possível. O comando `expirar_pontos` lança a expiração dos lotes vencidos. As telas de consulta não gravam nada: calculam o saldo só com os lotes ainda válidos.

P. Como o nível (Bronze/Prata/Ouro) é calculado?

> Pelos pontos **acumulados nos últimos 12 meses**, não pelo saldo (resgatar não rebaixa o cliente). O bônus da compra usa o nível de **antes** dela. Os pontos são calculados sobre o valor pago depois do desconto, sem centavos.

P. Como evitam que duas vendas simultâneas gastem o mesmo ponto?

> A venda roda em transação e trava a linha do cliente com `select_for_update`; a segunda venda espera a primeira terminar e já enxerga o saldo novo. Nas avaliações, além da trava, uma **restrição única no banco** garante uma avaliação por cliente e produto.

P. Os dados do dashboard são reais?

> São gerados pelo comando `popular_dados`, mas cada venda passa pelo **mesmo** `registrar_venda` do PDV, em ordem cronológica. Ele nunca grava pontos direto, então os números do dashboard obedecem às regras. O teste `test_telas.py` confere isso. A semente é fixa (`--semente`), então os dados são reproduzíveis.

4 PDV offline e idempotência

P. O que acontece se a internet do caixa cair?

> A venda vai para uma fila no `localStorage` (`fatec:vendas`) e é reenviada ao carregar a página e quando a conexão volta. A data original da venda vai junto, para os pontos e a validade contarem do dia certo.

P. E se a mesma venda for enviada duas vezes?

> O front gera um **UUID** para cada venda. O `POST /api/vendas/` é **idempotente** nesse id: um reenvio devolve 200 com a venda que já existe, sem duplicar pontos. Além disso, o preço sempre vem do banco: o caixa só manda SKU e quantidade, então não dá para alterar preço pelo navegador.

5 Animações no frontend

> **Resumo:** não usamos biblioteca de animação (como Framer Motion ou GSAP). As animações são **CSS puro** (`@keyframes` e `transition`) escritas com **Tailwind CSS v4**, e o React só decide *quando* trocar o conteúdo. Os gráficos animam sozinhos pela biblioteca **Recharts**, e os ícones são do **lucide-react**.

P. Como foi feita a vitrine de tênis da página inicial?

> A cada 3 segundos um tênis sai e outro entra, e o fundo da página assume a cor do estúdio da foto. Divisão de tarefas:
>
> - **React** (`hooks/useVitrine.ts`): um `setTimeout` troca o índice da foto atual e guarda a anterior, que ainda está saindo. A ordem das fotos é sorteada a cada visita.
> - **CSS** (`index.css`, classes `.landing-tenis-*`): a troca acontece em três tempos, sem sobreposição: o tênis sai (0–300 ms), o fundo muda de cor (300–650 ms) e o novo tênis entra (a partir de 650 ms). Isso evita ver o retângulo da foto sobre um fundo de outra cor.
> - A cor vem de **variáveis CSS** (`--h-fundo`, `--h-tinta`), então cabeçalho, texto e botões mudam juntos com uma única `transition`.
> - Um efeito leve de "flutuar" (`landing-tenis-flutua`) roda em loop, e o texto do topo entra em cascata (`landing-sobe`, com atraso calculado por `--i`).

P. Por que CSS e não uma biblioteca?

> - As animações são simples (entrar, sair, deslizar). Uma biblioteca aumentaria o pacote baixado sem necessidade.
> - Animamos só `transform` e `opacity`, que o navegador processa na GPU sem recalcular o layout: fica suave até em máquina fraca.
> - Tailwind v4 permite declarar animações próprias como tokens (`--animate-subir`, `--animate-pop`) e usá-las como classe (`animate-subir`) no dashboard.

P. Tiveram problemas de desempenho?

> Sim: decodificar uma foto grande no momento da troca fazia a animação travar. A solução foi **pré-carregar e decodificar** todas as fotos ao abrir a página (`new Image()` + `img.decode()` em `VitrineTenis.tsx`). Em efeitos ligados à rolagem (`useProgressoScroll`), a medição é agendada com `requestAnimationFrame` e o evento de scroll é `passive`, para medir no máximo uma vez por quadro.

P. E acessibilidade? Animação incomoda algumas pessoas.

> Respeitamos a preferência do sistema `prefers-reduced-motion`: com ela ativa, as animações são desligadas no CSS e a vitrine para no primeiro tênis. O painel "Aparência e acessibilidade" da barra lateral também tem a opção de reduzir movimento (atributo `data-movimento` no `<html>`), para quem não configurou isso no sistema. A faixa de fotos que corre sozinha pausa quando o mouse ou o foco do teclado está sobre ela.

P. Quais outras animações existem?

> Blocos do dashboard sobem em cascata ao abrir; modais e avisos (*toasts*) aparecem com um "pop" curto (160 ms); barra lateral e botões usam `transition`; os gráficos do dashboard (Recharts) animam o desenho das barras e linhas por padrão.

6 Testes e qualidade

Ferramenta	Onde	O que cobre
Django TestCase (unittest)	`backend/src/tests/`, 7 arquivos, 100+ testes	Regras puras, serviços com banco, API, conta do cliente, avaliações, telas
Vitest	ao lado de cada arquivo e `src/testes/`	Regras do front, formatação, CPF, CSV do dashboard, camada de API
TypeScript estrito (`tsc -b`)	todo o frontend	Erros de tipo e de contrato com a API na compilação
oxlint	todo o frontend	Lint rápido (regras de React, hooks, boas práticas)

P. Como vocês testam o backend?

> Com o `TestCase` do Django, que cria um banco PostgreSQL de teste e desfaz cada teste com rollback. Temos uma regra: **toda regra de `regras.py` e `services.py` tem um teste correspondente**. Exemplos: o bônus de 10% da Prata, resgate em blocos de 100 pontos, consumo FIFO, expiração em 12 meses, idempotência da venda, CPF da compra online vindo da sessão. Rodar: `docker compose exec backend python manage.py test`.

P. E o frontend, como testam sem o backend rodando?

> Com **Vitest** em ambiente Node. Testamos a lógica pura (`regras.ts`, `utils/`, CSV) e a camada de API: `src/testes/api.test.ts` troca o `fetch` por uma versão falsa (`testes/apiFalsa.ts`) que devolve respostas prontas (*mocks*) de `src/testes/dados/`. Detalhe importante: esses dados falsos são tipados com os tipos `Api...`, então se o backend mudar um campo, o `tsc` acusa o erro e somos obrigados a atualizar tipo, dado falso e teste juntos. Rodar: `npm test`.

P. Por que o front repete as regras de pontos (`regras.ts`)? Não fica duplicado?

> Fica, de propósito e limitado: serve só para mostrar a **prévia** dos pontos no caixa e na loja enquanto o cliente monta o carrinho, inclusive offline. O valor que vale é sempre o do backend. As duas versões mudam juntas, e cada uma tem seu teste (`regras.test.ts` e `test_regras.py`).

P. Há testes de componente ou ponta a ponta (E2E)?

> Não. Priorizamos as regras de negócio e o contrato da API, onde um erro custa dinheiro ou pontos. O próximo passo seria Playwright para os fluxos principais (venda no PDV, cadastro e compra online).

7 Segurança e LGPD

P. Como tratam o CPF dos clientes?

> Nas listas, ranking e perfil o CPF aparece **mascarado** (`cpf.mascarar`) e o cliente é identificado pelo `id` interno, não pelo CPF. A validação dos dígitos do CPF existe igual no back (`cpf.py`) e no front (`utils/cpf.ts`).

P. E o consentimento para receber ofertas?

> O cliente marca no perfil. Guardamos **a data** da autorização (`contato_autorizado_em`) como prova, como pede a LGPD: marcar de novo não muda a data e desmarcar apaga o consentimento.

P. O cliente pode manipular preço ou pontos pelo navegador?

> Não. O front manda só SKU, quantidade, forma de pagamento e quantos pontos quer usar. Preço, desconto, bônus e saldo são calculados e conferidos no servidor; uma violação de regra gera `RegraViolada` e resposta 400 com a mensagem de erro.

8 Limitações e próximos passos

P. O que vocês fariam com mais tempo?

> - Login da equipe na área da loja, com permissões por perfil (caixa, gerente).
> - Limite de tentativas de login e de cadastro (*rate limiting*).
> - Tela para o cliente trocar a senha (a API `POST /api/conta/senha/` já existe) e recuperação de senha.
> - Testes E2E com Playwright.
> - Agendar `expirar_pontos` automaticamente (cron) em produção.
> - HTTPS com cookies `Secure` e deploy fora do modo de desenvolvimento.

P. Onde vocês usaram IA no projeto?

> Todo trecho feito com ajuda de IA está marcado no código com um comentário `Feito com IA: ...` explicando o porquê, e o documento `docs/onde-usamos-ia.pdf` reúne esses pontos. A regra de negócio e as decisões foram revisadas pela equipe e são cobertas pelos testes.